



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине
«Распределенные СУБД»

Выполнил студент группы ИКБО-66-23

Смирнов А.Ю.

Принял ст. преподаватель

Гуличева А.А.

Москва 2026

Базовые сведения

База данных Cassandra – это распределенная NoSQL-СУБД, разработанная для обработки больших объемов данных с высокой доступностью и производительностью. Вот основные сведения:

Основные характеристики:

1. Распределенность:
 - Cassandra использует распределенную архитектуру без единой точки отказа;
 - данные автоматически реплицируются между узлами кластера.
2. Масштабируемость:
 - поддерживает линейное масштабирование: добавление новых узлов увеличивает производительность и объем хранимых данных.
3. Высокая доступность:
 - репликация данных обеспечивает отказоустойчивость;
 - подходит для систем, где критически важна бесперебойная работа.
4. Модель данных:
 - использует модель «столбец-семейство» (column-family), где данные хранятся в виде строк с гибкой схемой;
 - поддерживает JSON-подобные структуры.
5. Язык запросов:
 - использует CQL (Cassandra Query Language), похожий на SQL, но с ограничениями, связанными с распределенной природой системы.
6. Консистентность:
 - поддерживает настраиваемую консистентность (например, ONE, QUORUM, ALL), что позволяет балансировать между доступностью и точностью данных.
7. Производительность:
 - оптимизирована для записи: данные сначала записываются в журнал (commit log), а затем в memtable и SSTable.

Преимущества:

- отсутствие единой точки отказа;
- поддержка больших объемов данных;
- гибкость схемы данных;
- высокая производительность при записи и чтении.

Недостатки:

- ограниченная поддержка сложных запросов (например, JOIN);
- требует тщательной настройки для достижения оптимальной производительности.

Использование:

Cassandra применяется в системах, где важны масштабируемость и отказоустойчивость, например:

- социальные сети;
- IoT-платформы;
- рекомендательные системы;
- аналитика в реальном времени.

Cassandra активно используется компаниями, такими как Netflix, Apple и Instagram, для обработки больших объемов данных.

Модель данных

В терминологии кассандры приложение работает с пространством ключей (keyspace), что соответствует понятию схемы базы данных (database schema) в реляционной модели. В этом пространстве ключей могут находиться несколько колоночных семейств (column family), что соответствует понятию реляционной таблицы. В свою очередь, колоночные семейства содержат колонки (column), которые объединяются при помощи ключа (row key) в записи (row). Колонка состоит из трех частей: имени (column name), метки времени (timestamp) и значения (value). Колонки в пределах записи упорядочены. В отличие от реляционной БД, никаких ограничений на то, чтобы записи (а в терминах БД это строки) содержали колонки с такими же

именами, как и в других записях – нет. Также в последних версиях кассандры появилась возможность выполнять запросы определения и изменения данных (DDL, DML) при помощи языка CQL.

Особенности

Запись в кассандру работает с большей скоростью, чем чтение. Это меняет подход, который применяется при проектировании. Если рассматривать кассандру с точки зрения проектирования модели данных, то проще представить колоночное семейство не как таблицу, а как материализованное представление (materialized view) – структуру, которая представляет данные некоторого сложного запроса, но хранит их на диске. Вместо того, чтобы пытаться как-либо скомпоновать данные при помощи запросов, лучше постараться сохранить в колоночное семейство все, что может понадобиться для этого запроса. То есть, подходить необходимо не со стороны отношений между сущностями или связями между объектами, а со стороны запросов: какие поля требуются выбрать; в каком порядке должны идти записи; какие данные, связанные с основными, должны запрашиваться совместно – всё это должно уже быть сохранено в колоночное семейство. Количество колонок в записи ограничено теоретически 2 миллиардами.

Цель работы

Ознакомление с основами Apache Cassandra, ее архитектурными особенностями, моделью данных и практическими аспектами работы с ней.

Задачи:

1. Изучить основные характеристики Apache Cassandra, включая распределенность, масштабируемость, высокую доступность, консистентность и производительность.
2. Рассмотреть модель данных Cassandra (столбец-семейство) и язык запросов CQL.
3. Ознакомиться с процедурой установки и запуска контейнера Cassandra с использованием Docker.
4. Освоить базовые команды cqlsh для взаимодействия с Cassandra, включая создание пространств ключей (keyspaces), таблиц, а также операции записи и чтения данных.
5. Выполнить операции по манипуляции данными (INSERT, SELECT, DELETE, TRUNCATE) и управлению схемой (DROP TABLE) в Cassandra.

Описание выполнения задач

```
PS C:\Users\Redmi> docker run -d --name student-smirnov-ikbo-66-23 cassandra
Unable to find image 'cassandra:latest' locally
latest: Pulling from library/cassandra
4831516dd0cb: Pull complete
01d6644578cf: Pull complete
432bb378e3c3: Pull complete
6e747f38727c: Pull complete
fb884c5102dd: Pull complete
0d807ef6301a: Pull complete
93afc517788d: Pull complete
968557cbd4a4: Pull complete
4684065ca0cb: Download complete
58f44538882c: Download complete
Digest: sha256:2c74bccb038a57ec3d568ea3dd5353652f70efd8746b1901d73fdb7f372874e9
Status: Downloaded newer image for cassandra:latest
022f2b04ad3a73d7b289891d0b4150e206a0effe4db4cff6602bc49b178522f4
PS C:\Users\Redmi> docker run -d --name student-smirnov-ikbo-66-23 cassandra
docker: Error response from daemon: Conflict. The container name "/student-smirnov-ikbo-66-23" is already in use by container "student-smirnov-ikbo-66-23". You can reuse the name with the -f (or --force) flag.
Run 'docker run --help' for more information
PS C:\Users\Redmi> |
```

Рисунок 1 – запуск контейнера с образом Cassandra

```
PS C:\Users\Redmi> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
022f2b04ad3a   cassandra  "docker-entrypoint.s..." 8 minutes ago  Up 8 minutes  7000-7001/tcp, 7199/tcp, 9042/tcp, 9160/tcp  student-smirnov-ikbo-66-23
```

Рисунок 2 – проверьте работоспособность контейнера

```
PS C:\Users\Redmi> docker exec -ti student-smirnov-ikbo-66-23 cqlsh
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> |
```

Рисунок 3 – получение доступа в оболочку для работы с CQL – языком запросов в Cassandra

```
cqlsh> DESCRIBE CLUSTER;

Cluster: Test Cluster
Partitioner: Murmur3Partitioner
Snitch: DynamicEndpointSnitch
```

Рисунок 4 – информация о текущем кластере, в котором мы работаем

```
cqlsh> DESCRIBE KEYSPACES;
```

system	system_distributed	system_traces	system_virtual_schema
system_auth	system_schema	system_views	

Рисунок 5 – информация о пространствах ключей, имеющихся в кластере

```
cqlsh> CREATE KEYSPACE my_keyspace WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> DESCRIBE KEYSPACES;
```

Рисунок 6 – создание пространства ключей с указанием стратегии репликации и коэффициентом репликации

```
cqlsh> DESCRIBE KEYSPACES;
```

my_keyspace	system_auth	system_schema	system_views
system	system_distributed	system_traces	system_virtual_schema

Рисунок 7 – информация о пространствах ключей, имеющихся в кластере, включая наше новое пространство ключей

```
cqlsh> USE my_keyspace;
cqlsh:my_keyspace> |
```

Рисунок 8 – переключение на пространство ключей my_keyspace

```
cqlsh> USE my_keyspace;
cqlsh:my_keyspace> CREATE TABLE user (first_name text, last_name text, PRIMARY KEY(first_name));
cqlsh:my_keyspace>
```

Рисунок 9 – создание таблицы «user» с двумя столбцами для хранения имени и фамилии, оба типа text

```
cqlsh:my_keyspace> INSERT INTO user (first_name, last_name) VALUES ('BILL', 'Nguyen');
```

Рисунок 10 – запись значения в таблицу

```
cqlsh:my_keyspace> SELECT * FROM user WHERE first_name='BILL';
```

first_name	last_name
BILL	Nguyen

(1 rows)
cqlsh:my_keyspace>

Рисунок 11 – чтение значения из таблицы

```
cqlsh:my_keyspace> DELETE last_name FROM user WHERE first_name='BILL';
```

Рисунок 12 – удаление столбца из строки с заданным ключом

```
cqlsh:my_keyspace> SELECT * FROM user WHERE first_name='BILL';  
  
first_name | last_name  
-----+-----  
BILL      | null  
  
(1 rows)  
cqlsh:my_keyspace> |
```

Рисунок 13 – чтение значения из таблицы

```
cqlsh:my_keyspace> DELETE FROM user WHERE first_name='BILL';
```

Рисунок 14 – удаление строки с заданным ключом

```
cqlsh:my_keyspace> SELECT * FROM user WHERE first_name='BILL';  
  
first_name | last_name  
-----+-----  
  
(0 rows)  
cqlsh:my_keyspace> |
```

Рисунок 15 – чтение значения из таблицы

```
cqlsh:my_keyspace> TRUNCATE user;
```

Рисунок 16 – удаления всех данных из таблицы, но
сама таблица и её схема остаются неизменными

```
cqlsh:my_keyspace> DROP TABLE user;
```

Рисунок 17 – полное удаление таблицы, включая её схему и все
данные

Вывод

В ходе выполнения практической работы я познакомился с основами работы с Apache Cassandra. С помощью Docker я развернул контейнер с базой данных, научился подключаться к ней через cqlsh и опробовал основные команды CQL: создание пространства ключей и таблиц, добавление, чтение и удаление данных. Главное, что я понял — в Cassandra всё строится вокруг запросов, а не связей между таблицами, как в обычных базах данных. Первичный ключ здесь нужно выбирать очень внимательно, потому что потом его не изменить. В целом, работа помогла разобраться, как устроена эта система и с чем её едят.

Главный вывод, который я сделал — в Cassandra всё завязано на первичном ключе. Он не только уникально идентифицирует запись, но и определяет, как данные будут распределяться по кластеру и в каком порядке храниться. Поэтому выбирать его нужно с умом — изменить потом не получится. Работа помогла мне понять базовые принципы этой системы и получить практические навыки, с которых можно начинать дальнейшее изучение Cassandra.

Ответы на контрольные вопросы

1. Сходства и различия моделей данных в реляционных СУБД и в Cassandra.

Сходства:

- В обеих моделях данные организованы в таблицы
- Используется понятие первичного ключа для уникальной идентификации записей
- Язык запросов CQL во многом напоминает SQL (SELECT, INSERT, DELETE, UPDATE)
- Поддерживаются типы данных, индексы и вторичные индексы

Различия:

- Отсутствие связей между таблицами: В Cassandra нет JOIN, внешних ключей и транзакций ACID. Все связи нужно реализовывать на уровне приложения
- Гибкая схема данных: В реляционных БД все строки таблицы имеют строго одинаковый набор столбцов. В Cassandra разные строки в одной таблице могут содержать разный набор колонок
- Денормализация: В реляционных БД мы стремимся к нормализации и устранению избыточности. В Cassandra наоборот — данные часто хранятся с избыточностью, чтобы сразу отдавать их в том виде, в котором они нужны для запросов
- Проектирование: Реляционные БД проектируются от сущностей и связей между ними. Cassandra проектируется от запросов — сначала думаем, какие запросы будут выполняться, потом создаём таблицы, которые максимально быстро отдадут данные для этих запросов
- Производительность: Реляционные БД обычно сбалансированы по чтению и записи. Cassandra сильно оптимизирована на запись — она пишет данные значительно быстрее, чем читает

2. Для каких задач может использоваться Cassandra?

Cassandra применяется в проектах, где критически важны масштабируемость, отказоустойчивость и высокая скорость записи больших объёмов данных:

- Социальные сети и мессенджеры — хранение сообщений, постов, лайков (Instagram)
- IoT-платформы — запись миллионов событий с датчиков и устройств
- Системы управления сессиями — хранение пользовательских сессий с отказоустойчивостью
- Электронная коммерция — каталоги товаров, корзины покупок, особенно при пиковых нагрузках
- Финансовые системы и биллинг — хранение транзакций и истории платежей